

Final Project Structure

```
Final_Project_repo/
├── .github/
│   └── workflows/
│       └── ci_cd.yml                # GitHub Actions CI/CD workflows (Build, Test, Deploy)
├── config/
│   └── project_config.yaml          # Environment variables, feature list, hyperparameter defaults
├── data/
│   └── raw/                         # Placeholder for initial raw data/schemas
├── docker/
│   ├── airflow/
│   │   └── Dockerfile               # Custom Airflow image (for adding custom dependencies)
│   ├── model_api/
│   │   └── Dockerfile               # Dockerfile for the prediction API service
│   └── requirements.txt              # Base Python dependencies for all services
├── infrastructure/
│   ├── docker-compose.yml           # Orchestrates Airflow, MLflow, Postgres, and Superset
│   ├── postgres/
│   │   └── init.sql                 # SQL scripts for creating database, tables, and roles
│   ├── superset/
│   │   └── config.py                # Custom Superset configuration (DB connections, security)
├── mlflow_artifacts/                 # Directory for local MLflow tracking server persistence
├── notebooks/
│   └── EDA.ipynb                    # Exploratory Data Analysis (EDA) and initial model development
├── src/
│   ├── __init__.py
│   ├── data/
│   │   ├── ingestion.py             # Logic for extracting data into PostgreSQL
│   │   └── features.py              # Feature engineering logic (lagging, temporal, external data blending)
│   ├── models/
│   │   └── train.py                  # Model training, hyperparameter tuning, and MLflow logging
│   ├── api/
│   │   └── server.py                 # FastAPI/Flask app to serve predictions (loads model from MLflow)
│   ├── monitoring/
│   │   └── drift_detector.py         # Logic for calculating model/data drift metrics
├── airflow_dags/
│   ├── data_pipeline_dag.py          # Orchestrates Data Ingestion and Feature Engineering
│   ├── training_pipeline_dag.py      # Orchestrates Model Training, Evaluation, and MLflow promotion
│   └── inference_pipeline_dag.py     # Orchestrates prediction, results storage, and monitoring
└── README.md
```

Folder Descriptions

- **.github/workflows:** Contains the YAML files that define your automated pipelines for Continuous Integration and Continuous Deployment (CI/CD) using GitHub Actions.
- **config:** Stores non-code configuration files, allowing easy parameter adjustment without modifying source code.
- **data:** Holds static data files, schemas, or seed data necessary for the initial setup.
- **docker:** Centralizes all files related to Docker images, ensuring reproducible environments for training, deployment, and Airflow tasks.
- **infrastructure:** This is the heart of your MLOps stack, defining how all the separate services (Airflow, MLflow, Postgres, Superset) are run together using docker-compose.yml.
- **src:** Contains all modular, production-ready Python code. This code is imported and executed by Airflow DAGs.
- **airflow_dags:** Contains the Python files that define the Airflow DAGs, which orchestrate the entire end-to-end pipeline using the code defined in `src/`.

End-to-End MLOps Pipeline: Load Forecasting of Meter Data

Project Goal

To create a fully automated and production-ready Machine Learning system that predicts future electricity or resource load based on historical meter data, using an MLOps approach for reproducibility, monitoring, and continuous improvement.

Technology Stack

Component	Tool Used	Purpose
Data Storage	PostgreSQL	Relational database for storing raw meter data, processed features, and model monitoring metrics.
Orchestration	Apache Airflow	Defining, scheduling, and monitoring the ETL, Training, and Validation DAGs (Directed Acyclic Graphs).
Experiment Tracking	MLflow	Logging experiments, metrics, parameters, and versioning models.
Containerization	Docker	Packaging the Python environments for data processing, training, and model serving to ensure consistency.
CI/CD	GitHub Actions	Automating the build, test, and deployment of Docker images and Airflow DAGs.
Visualization/Monitoring	Apache Superset	Creating real-time dashboards for model performance (drift, metrics) and business metrics (forecast vs. actual load).

Phase 1: Setup and Infrastructure (IaC)

- Repository Setup (GitHub):** Create the primary repository with standard directories (data/, dags/, src/, models/, config/).
- Containerization (Docker):**
 - Create a docker-compose.yml file to orchestrate all services: PostgreSQL, Airflow (Webserver, Scheduler, Worker), MLflow Tracking Server, and Superset.
 - Define custom Dockerfiles for the training and serving environment, ensuring all Python dependencies (e.g., pandas, scikit-learn, mlflow) are pinned.
- Database Provisioning (PostgreSQL):**
 - The docker-compose.yml initiates a Postgres container.
 - A schema script creates necessary tables: raw_meter_data, features, model_metrics, forecast_results.

Phase 2: Data Pipeline (ETL/Feature Engineering)

Tool: Apache Airflow

1. **Data Ingestion DAG (data_ingestion_dag.py):**
 - **Task 1 (Extract):** Connects to the source system (e.g., internal data lake, API, or CSV files).
 - **Task 2 (Load):** Ingests raw meter data (timestamp, meter ID, consumption/load) into the PostgreSQL raw_meter_data table.
2. **Feature Engineering DAG (feature_engineering_dag.py):**
 - **Task 1 (Cleanup):** Handles missing values, outliers, and time-series resampling.
 - **Task 2 (Feature Creation):** Generates time-series features essential for load forecasting:
 - Lagged load values.
 - Temporal features (hour of day, day of week, month, year).
 - Weather/Calendar features (holidays, temperature data, if available).
 - **Task 3 (Store):** Saves the final, prepared features into the PostgreSQL features table.

Phase 3: Model Training and Experimentation

Tools: Apache Airflow, MLflow, Docker

1. **Training DAG (model_training_dag.py):**
 - **Task 1 (Pull Data):** Fetches the latest features from the features table in PostgreSQL.
 - **Task 2 (Model Train - DockerOperator):**
 - This task uses an Airflow DockerOperator to run the training script (src/train.py) inside the isolated training Docker image.
 - The script logs all parameters (e.g., hyperparameters, training data version), performance metrics (e.g., RMSE, MAE), and the serialized model artifact to the **MLflow Tracking Server**.
 - **Task 3 (Model Evaluation & Promotion):** Compares the new model's performance against the currently staged model in the **MLflow Model Registry**. If the new model is superior (e.g., lower RMSE), it is transitioned to the "Staging" or "Production" phase in the registry.

Phase 4: CI/CD and Deployment (Model Serving)

Tools: GitHub Actions, Docker, MLflow

1. **Continuous Integration (CI):**
 - **Trigger:** A pull request (PR) or push to the main branch.
 - **GitHub Action 1 (Build & Test):**
 - Runs unit tests on all Python scripts (src/).
 - Lints and validates the Airflow DAG syntax.
 - Builds the Model Serving Docker Image (contains the prediction API code, e.g., using FastAPI).
 - Tags the image with the Git commit SHA.
2. **Continuous Deployment (CD):**

- **Trigger:** Merge to the main branch, or a manual tag indicating a production release.
- **GitHub Action 2 (Deploy):**
 - Pushes the Model Serving Docker Image to a container registry (e.g., Docker Hub).
 - Updates the production environment (e.g., Kubernetes or a cloud container service) to pull and run the latest Model Serving image. The serving API uses the **MLflow Model Registry** to dynamically load the model marked as "Production."

Phase 5: Production and Monitoring

Tools: Apache Airflow, Apache Superset, PostgreSQL

1. **Inference DAG (model_inference_dag.py):**
 - **Task 1 (Get Features):** Fetches the latest input data (time features, weather forecasts) from PostgreSQL.
 - **Task 2 (Predict - DockerOperator):** Calls the deployed Model Serving API to generate load forecasts.
 - **Task 3 (Store Results):** Saves the predictions into the PostgreSQL forecast_results table.

2. **Visualization (Apache Superset):**
 - Superset connects directly to the PostgreSQL database.
 - **Dashboards created:**
 - **Operational Dashboard:** Visualizing the actual load, forecasted load, and prediction error over time (using data from forecast_results).
 - **Model Health Dashboard:** Displaying key metrics (RMSE, MAE) and drift indicators (using data from model_metrics).